DV Track

Lab T-DV1: Layered Class-Based Testbench

Generator, Driver, Monitors and a Model-Based Scoreboard

ASIC Design, Verification and Implementation Onboarding

Kerim Turak

October 18, 2026

Lab objective: Build a layered, object-oriented testbench for the data cache around reusable `mem_if` driver and monitor classes, connect the Lab T-MOD cache model to the RTL through a model-based scoreboard, and measure how much better it finds bugs than the Lab V3 testbench.

Duration	2 weeks (approximately 35–40 hours)
Prerequisites	Labs V3 and T-MOD
Designs	`dcache` with `arb_mem` (unchanged), verified by a new environment; `cache_model_pkg` (reused unchanged); `mem_if` verification IP (new)
Reused in	T-DV2 (every component maps to a UVM class; the `mem_if` classes become a UVM agent)

Contents

1 Overview **3**

2 Testbench Architecture **4**

2.1 Layers . . . 4

2.2 Components . . . 5

2.3 Communication . . . 6

2.4 Reusable `mem_if` Verification IP . . . 6

3 Connecting the Model to the RTL **7**

3.1 Scoreboard Algorithm . . . 7

3.2 From Three Checks to One Model . . . 8

3.3 Ending the Test . . . 9

4 Tests and the Blueprint Pattern **9**

5 Reusing the Lab V3 Coverage **11**

6 Bug-Injection Experiment **11**

7 Requirements / Tasks **12**

8 Repository and Flow **13**

9 Deliverables **14**

10 Assessment **15**

11 Review Questions **15**

12 References **15**

1 Overview

In Lab V3 the testbench checked the cache with three separate pieces: a flat memory model for read data, a tag model for hit/miss, and a final memory comparison after flush. Each piece knew only a part of the cache's behavior, and the stimulus, the checks and the coverage were tasks and classes mixed inside one testbench module. That is fine for one block and one engineer. It does not scale to a team that verifies many blocks sharing the same protocols, where a driver written once must serve every block that speaks `mem_if`, and where a new test must not require editing the environment.

In this lab you replace the three checks with one **reference model** — the transaction-level cache model you built and verified in Lab T-MOD — and you build a **layered testbench** from classes around it: generator, driver, monitors, scoreboard, environment and test. For every CPU request the model predicts the result, the read data and the exact list of memory transfers, and the scoreboard compares all of them with what the RTL did. This is the structure of a UVM testbench, built by hand. When you meet UVM in Lab T-DV2, you will recognize every part.

Where this lab sits. It builds on Lab V3 (the `dcache` test setup, tests, coverage plan and bug log) and Lab T-MOD (object-oriented SystemVerilog and `cache_model_pkg` in `dv/model/cache_model/`, already verified on its own). Neither is re-taught here: when this document says `access()`, `flush()` or `clone()`, it means the T-MOD API. The output of this lab — the class-based environment in `hw/ip/dcache/dv_layered/` and the `mem_if` classes in `dv/vip/mem_if/` — is translated into UVM in Lab T-DV2.

By the end of this lab you should be able to:

1. Partition a testbench into layers (test, environment, components, transactions, interfaces) and justify where each responsibility lives.
2. Write protocol verification IP (driver and monitor classes) that knows nothing about the design it is used with.
3. Choose between mailboxes, method calls, events and semaphores for communication between components, and explain the choice.
4. Connect a transaction-level reference model to cycle-level RTL through a scoreboard, including transfers that happen before the request that causes them is accepted.
5. End a test cleanly: drain stimulus, flush, compare, report.
6. Change the stimulus of a test without editing the generator, using the blueprint pattern and polymorphism.
7. Measure the bug-finding power of a testbench with bug injection and compare two testbenches objectively.

2 Testbench Architecture

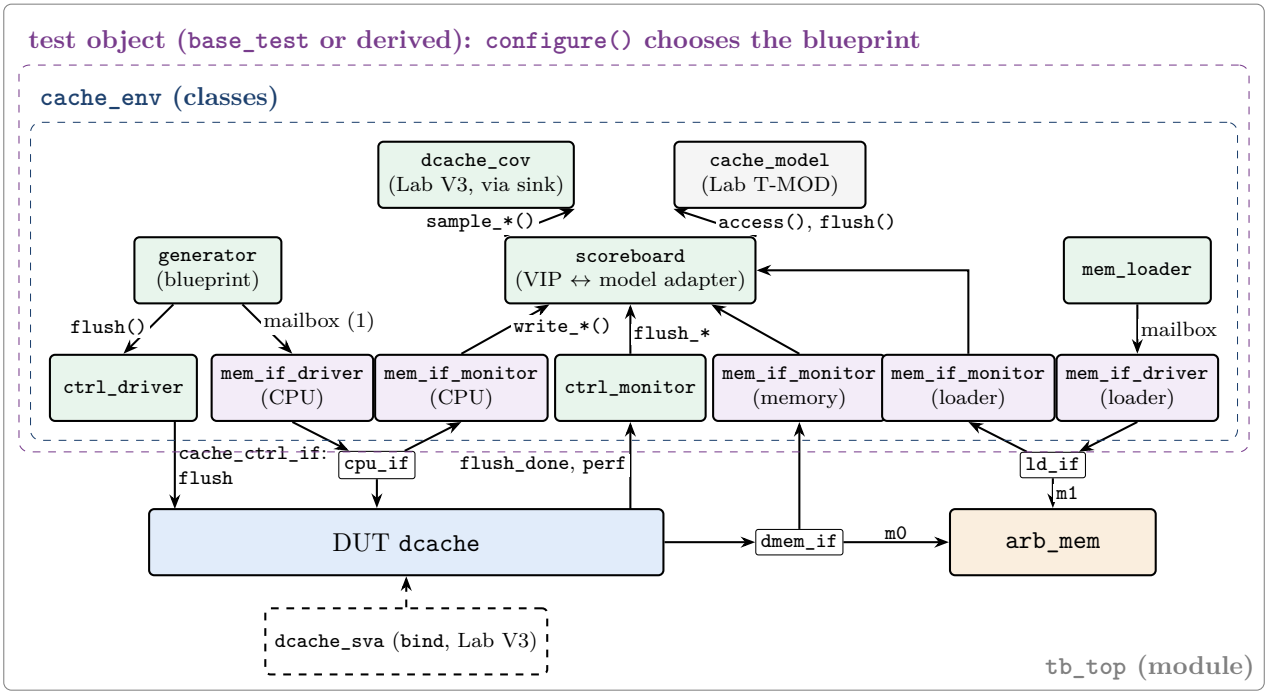


Figure 1: Architecture of the layered testbench. Purple: classes of the reusable `mem_if` verification IP (`dv/vip/mem_if/`); green: cache-specific environment classes; grey: the Lab T-MOD model, used unchanged; blue/orange: the DUT and the memory. Solid arrows into the scoreboard are subscriber method calls; the arrows to the pins are virtual interfaces.

Figure 1 shows the layers and every connection. Purple: classes from the reusable `mem_if` verification IP in `dv/vip/mem_if/`. Grey: the Lab T-MOD model, used unchanged. The loader is a second `mem_if_driver` instance on port `m1`, wrapped by a small class that knows the preload pattern. The control signals of the cache (`flush`, `flush_done`, `perf`) are not part of `mem_if`; they reach the environment through a small interface, `cache_ctrl_if`, declared in `dv_layered/tb/` (Lab T-DV2 reuses it for its control agent): a small `ctrl_driver` pulses `flush` for the generator, and a `ctrl_monitor` tells the scoreboard when a flush starts and ends, with the counters sampled at `flush_done`. A third `mem_if_monitor` on the loader port lets the scoreboard check the preload and the final read-back too.

2.1 Layers

Layer	Contents	Changes when
Test	Test classes: configuration, blueprint, knobs	A new scenario is needed
Environment	`cache_env`: creates, connects, starts, ends	The DUT gets a new interface
Components	Generator, drivers, monitors, scoreboard, coverage	A protocol or a check changes
Transactions	`mem_if_item` and derived stimulus classes; model transactions (T-MOD)	A field is added
Signal	`tb_top`: clock, reset, interfaces, DUT, `bind`	The DUT's ports change

The point of the layering is the right-hand column: each kind of change touches one layer only. A test never touches a virtual interface; a driver never knows which test is running; the model never knows a clock exists.

2.2 Components

Class	Responsibility
`cache_cfg` (T-MOD)	Run-time geometry (lines, words per line, ways, policy name), set from the `tb_top` parameters; plus the testbench knobs (number of transactions, verbosity) in a derived `cache_tb_cfg`
`mem_if_item` (VIP)	One `mem_if` transaction: `we`, `addr`, `wdata`, `idle`, and `rdata` filled by the monitor; `clone()`, `compare()`, `convert2string()`
`dcache_txn`, `conflict_txn`, ...	Stimulus classes derived from `mem_if_item`, with the knobs and constraints of the Lab V3 `cache_txn` family
`generator`	Produces CPU transactions from a *blueprint* object and puts them into a mailbox
`mem_if_driver` (VIP)	Gets items from a mailbox and drives them through a `mem_if` virtual interface, honoring idle cycles and the handshake
`mem_if_monitor` (VIP)	Observes accepted requests and read responses on one `mem_if`; calls its subscriber
`mem_loader`	Preload and final read-back on port `m1`, using a `mem_if_driver` and a `mem_if_monitor`
`scoreboard`	Owns the model; converts observations to model transactions; compares predictions with observations
`dcache_cov` (Lab V3)	Coverage model, sampled by the scoreboard
`cache_env`	Creates and connects all components, starts them, ends the test cleanly
`base_test` and derived tests	Choose the configuration and the blueprint; one class per test

All components that run in time extend an abstract base class. Because the VIP classes extend it too, it is added to the shared `dv/common/tb_utils_pkg.sv` (Lab V1), not to a cache-specific package. The environment holds them in a queue of `component` handles and starts them with one loop, without knowing their concrete types:

```
virtual class component;
  string name;
  function new(string name); this.name = name; endfunction
  pure virtual task run();
  virtual function void report(); endfunction   // optional end-of-test summary
endclass

// in cache_env::run()
foreach (comps[i]) begin
  automatic int k = i;              // loop-variable pitfall, Lab V2
  fork comps[k].run(); join_none
end
```

2.3 Communication

Mechanism	Behavior	Use in this lab
`mailbox #(T)`	Typed FIFO between processes; `put()` and `get()` block; a *bounded* mailbox blocks the producer when full	Generator → driver. Bound it to 1 so the generator cannot run ahead of the DUT
Method call	The producer calls a function of the consumer immediately; no blocking, no buffering	Monitors → scoreboard (like a UVM analysis port)
`event`	`->ev` triggers; `@ev` or `wait(ev.triggered)` waits	Generator done; flush done; end of test
`semaphore`	A pool of keys; `get()` waits for a key	Optional: protect the loader port if two processes may use it

Explain in `doc/verification.md` why the monitor-to-scoreboard path uses method calls and the generator-to-driver path uses a bounded mailbox.

Tool note. The testbench relies on Verilator's support for classes, parameterized classes, `mailbox`, `semaphore`, virtual interfaces in classes and constrained randomization. The coordinator confirms the installed version before the lab starts. The `randomize()` fallback of Lab V3 (`my_randomize()`) applies unchanged.

On the Synopsys cloud: VCS runs this environment as it is: `make -C hw/ip/dcache sim SIM=vcs DV_ENV=dv_layered TEST=random`. VCS supports every construct used here (parameterized classes, mailboxes, virtual interfaces in classes, `dist` and inline `randomize() with`, covergroups) and is four-state, so an uninitialized cache word shows up as `x` in a mismatch message instead of a plausible 0. Verilator 5.013 (installed on the cloud, below the program minimum 5.020) has no constraint solver for `dist`/`with` and cannot run the Lab V3 or T-DV1 stimulus; use `SIM=vcs` there. Waves: `WAVES=1`, then `make waves` opens Verdi on the FSDB; Verdi also shows class objects and their fields when the model is built with `-kdb -debug_access+all` (done by `WAVES=1`).

2.4 Reusable `mem_if` Verification IP

Every `mem_if` in the system — CPU side, memory side, loader — speaks the same protocol (Lab R3). The driver and monitor therefore belong to the *protocol*, not to the cache, and live in the shared `dv/vip/mem_if/` directory as one package, `mem_if_vip_pkg`. The same monitor class is instantiated twice in this environment (CPU and memory side); the same driver class twice (CPU and loader). The package must not import anything cache-specific; `arb_mem`'s Lab V2 environment could use it unchanged.

```
// dv/vip/mem_if/mem_if_vip_pkg.sv  (excerpt)
package mem_if_vip_pkg;

  class mem_if_item #(int AW = 10, int DW = 32);
    rand bit            we;
    rand logic [AW-1:0] addr;
    rand logic [DW-1:0] wdata;
    rand int unsigned   idle;
    logic      [DW-1:0] rdata;            // filled in by the monitor
    constraint c_idle { idle dist { 0 := 70, [1:3] :/ 30 }; }  // ':/' spreads 30 over
    1..3
    virtual function mem_if_item #(AW, DW) clone();
    virtual function bit    compare(mem_if_item #(AW, DW) rhs);
    virtual function string convert2string();
```

```
  endclass

  // Anything that wants to receive monitor output implements this
  virtual class mem_if_subscriber #(int AW = 10, int DW = 32);
    pure virtual function void write_req(mem_if_item #(AW, DW) t);  // at acceptance
    pure virtual function void write_rsp(mem_if_item #(AW, DW) t);  // read completed
  endclass

  class mem_if_monitor #(int AW = 10, int DW = 32) extends component;
    virtual mem_if #(.AW(AW), .DW(DW)) vif;
    mem_if_subscriber #(AW, DW)        subs [$];   // any number of subscribers
    // run(): at every posedge, publish completed reads first, then the
    // acceptance (req_valid && req_ready) of this edge
  endclass

  class mem_if_driver #(int AW = 10, int DW = 32) extends component;
    virtual mem_if #(.AW(AW), .DW(DW))  vif;
    mailbox #(mem_if_item #(AW, DW))    in_mb;
    bit                                 idle_q;  // true while no item pending
    // run(): the Lab V2 driver task, now a method; drives with <=
  endclass
endpackage
```

Two design decisions to explain in your documentation:

- **Subscriber list instead of a scoreboard handle.** The monitor calls `write_req()` on every element of `subs`. It does not know that a scoreboard exists; adding a coverage collector or a protocol statistics class is one `push_back`. This is the hand-built version of a UVM analysis port.
- **Response before acceptance.** When a read response and a new acceptance happen at the same edge, the monitor publishes the response first. Otherwise the scoreboard would push a new expected value before popping the old one — harmless with a queue, but wrong for any subscriber that keeps "the current transaction".

The VIP item is `mem_if_item`; the model's transactions (`cpu_txn`, `mem_txn`) come from `cache_model_pkg`. Neither package imports the other. The **scoreboard is the adapter**: it converts a `mem_if_item` observed on the CPU side into a `cpu_txn` for `model.access()`, and the model's predicted `mem_txn`s into the form observed on the memory side. Keeping the conversion in one place is what lets the VIP, the model and the environment evolve independently.

3 Connecting the Model to the RTL

The `tb_top` module of this environment instantiates the Lab V3 DUT setup unchanged: `dcache`, `arb_mem`, three `mem_if` instances, the `dcache_sva` bind from `hw/ip/dcache/dv/sva/` (the design assertions stay active), plus `cache_ctrl_if`. It creates the test object and passes it the virtual interfaces; from then on, everything happens in classes.

The model's backing memory is initialized with the same preload pattern that the loader writes, and the model's `cache_cfg` is built from the `tb_top` parameters, so model and RTL have the same geometry. Print both at the start of the test; a geometry mismatch produces confusing errors thousands of transactions later.

3.1 Scoreboard Algorithm

Memory transfers for a miss happen *before* the CPU request is accepted, because the cache accepts a request only when it hits. The scoreboard therefore collects memory-side observations and compares

them when the CPU request is accepted:

1. The memory-side monitor calls `write_req(t)` for every accepted memory-side request. The scoreboard appends it to `observed_mem[$]`.
2. The CPU-side monitor calls `write_req(t)` for every accepted CPU request. The scoreboard converts it, calls `model.access()` and compares:
   - the predicted memory transactions with `observed_mem`, in order (address, direction and write data), then clears `observed_mem`;
   - for a read, it pushes the predicted data into `exp_rdata[$]`.
3. The CPU-side monitor calls `write_rsp(t)` for every read response; the scoreboard pops `exp_rdata` and compares.
4. On a flush, the scoreboard calls `model.flush()` and compares the write-backs observed until `flush_done` *as a set*, using an associative array indexed by address, because the specification does not fix the order in which lines are flushed.
5. At the end of the test, the DUT performance counters (`perf` via `cache_ctrl_if`) must equal the model's statistics exactly.

```
// The scoreboard receives both monitors; it tells them apart by a tag
// given to each subscriber adapter at construction (CPU or MEM side).
function void scoreboard::cpu_accept(item_t t);
  cpu_txn_t  c = to_cpu_txn(t);             // adapter: VIP -> model
  pred_t     p = model.access(c);           // T-MOD API
  if (p.mem_ops.size() != observed_mem.size())
    err($sformatf("%s: expected %0d memory transfers, observed %0d",
                  c.convert2string(), p.mem_ops.size(), observed_mem.size()));
  else foreach (p.mem_ops[i])
    if (!same_transfer(p.mem_ops[i], observed_mem[i]))
      err($sformatf("transfer %0d: exp %s obs %s", i,
                    p.mem_ops[i].convert2string(),
                    observed_mem[i].convert2string()));
  observed_mem.delete();
  if (!t.we) exp_rdata.push_back(p.rdata);
  cov.sample_access(t.we, to_result_e(p.result), idx_of(t.addr), off_of(t.addr));
endfunction
```

The field names of the prediction (`mem_ops`, `rdata`, `result`) are those of your Lab T-MOD API. If the scoreboard needs something the model does not provide, add it to the scoreboard, never to the model: the model is shared with the architecture-modeling track and is verified independently.

3.2 From Three Checks to One Model

Lab V3 check	Replaced by
Read data check (flat memory)	Predicted read data from the model, compared at the response
Hit/miss check (tag model)	Number, direction and order of predicted memory transfers, compared at acceptance
Victim and refill address check	Address and write data of every predicted transfer
Final memory check	Flush comparison as a set, plus the read-back of the whole memory against the model's backing memory
Perf counter check	Model statistics

The model-based scoreboard is strictly stronger: it compares the *write data* of every eviction at the moment it happens. Explain in `doc/verification.md` which bugs only the new scoreboard can find, for example a write-back with wrong data to an address that is later overwritten before the final memory check.

3.3 Ending the Test

1. The generator triggers `gen_done` after its last transaction.
2. The environment waits until the CPU driver is idle and the scoreboard has no pending expectations (`exp_rdata.size() == 0`, `observed_mem.size() == 0`).
3. The environment requests a flush through `cache_ctrl_if`, waits for `flush_done`, and lets the scoreboard compare.
4. The loader reads back the memory; it must equal the model's backing memory.
5. Every component's `report()` runs; the test prints `TEST PASSED` only if all of them are clean, using `tb_utils_pkg` (Lab V1).

A watchdog (`tb_utils_pkg`) ends the test with an error if any step hangs. A test that ends at step 1 would miss every bug in the last transactions and in the flush; a test that waits on a fixed delay instead of step 2 would be either slow or flaky.

4 Tests and the Blueprint Pattern

The generator owns a *blueprint*: one transaction object that it randomizes and clones for every new transaction. A test changes the stimulus by replacing the blueprint with an object of a derived class. The generator code never changes.

```
class generator #(int AW = 10, int DW = 32) extends component;
  mem_if_item #(AW, DW)            blueprint;
  mailbox #(mem_if_item #(AW, DW)) gen2drv;     // bounded to 1
  int unsigned                     num_txn;
  event                            done;

  virtual task run();
    repeat (num_txn) begin
      if (!blueprint.randomize()) $fatal(1, "randomize failed");
      gen2drv.put(blueprint.clone());   // clone: never share the blueprint
    end
    -> done;
  endtask
endclass

class conflict_test extends base_test;
  virtual function void configure();
    conflict_txn_t c = new();
    c.hot_index = 3;
    env.gen.blueprint = c;     // polymorphism: a derived object in a base handle
  endfunction
endclass
```

For `clone()` to return a `conflict_txn` and not a plain `mem_if_item`, every derived transaction class must override `clone()`. Show in `doc/verification.md` what happens if it does not: which constraint disappears, and which check or coverage bin tells you.

`base_test` defines the sequence of a test as non-virtual `run()` calling virtual hooks — `configure()`, `start_env()`, `stimulus()`, `wait_done()`, `end_test()` — so a derived test overrides only what it changes. Directed tests (`smoke`, `sequential`, `flush`) override `stimulus()` and put hand-made items into the mailbox (through the generator, which also owns the flushes) instead of running the generator loop.

Test term — Directed test. A hand-written sequence of accesses with a known expected result, aimed at one feature (here: hit, clean miss, dirty miss, write miss; or a flush with 0, 1 and all lines dirty). It is easy to debug but checks only what its author thought of; in this lab the directed tests are the first to run when the environment is new, because a failure points straight at the access that caused it.

The top module selects the test from `+TEST=` with a simple `case` statement that creates the matching test object. (UVM replaces this with a factory, Lab T-DV2.)

```
base_test t;
initial begin
  string name = "smoke";
  void'($value$plusargs("TEST=%s", name));
  case (name)
    "smoke":      begin smoke_test      s = new(); t = s; end
    "conflict":   begin conflict_test   c = new(); t = c; end
    // ... one line per test
    default:      $fatal(1, "unknown TEST=%s", name);
  endcase
  t.set_vifs(cpu_if, mem_if_m0, ld_if, ctrl_if);
  t.run();
end
```

Required tests: `smoke`, `sequential`, `fits`, `conflict`, `random` and `flush` from Lab V3, rewritten as test classes with the same plusargs (`NUM_TXN`, `WR_PCT`, `IDLE_PCT`, `ADDR_RANGE`, `FLUSH_EVERY`).

Test term — Smoke test. The shortest test that touches every basic behavior once — here ten directed accesses: cold miss, hit, write hit, dirty miss, refetch, write miss, write dirty miss. It answers "are the environment, the model and the RTL alive and in agreement?" in a second. A pass proves little; a failure means nothing else is worth running. It is the first test of the regression and of `make check-all`.

Test term — Constrained-random test. Stimulus drawn by `randomize()` from classes whose legal ranges and distributions are given by constraints (`fits`, `conflict`, `random`); every run prints its seed, so a failure is reproducible. Here the blueprint pattern decides *which* constraints apply without touching the generator. A pass with many seeds, checked by the model, is what finds the bugs nobody wrote a directed test for.

Test	Kind	What it is meant to show
`smoke`	directed	the Lab R4 cases (hit, clean/dirty miss, write miss) through the whole environment
`sequential`	directed	stride-1 reads over 4× the capacity: one clean miss per line, hit rate $1 - 1/L$
`fits`	constrained-random	accesses inside the capacity: only cold misses, write hits set dirty without traffic
`conflict`	constrained-random	`conflict_txn` blueprint: one index, `NUM_TAGS` tags; conflict misses and dirty evictions back to back
`random`	constrained-random	the whole memory with a flush every `FLUSH_EVERY` accesses: everything mixed, long runs
`flush`	directed + random	flush with 0, 1, some and all lines dirty, reads after each flush, random traffic with flushes

The replacement-policy test is model-only and belongs to Lab T-MOD. If you built the 2-way cache in Lab R4, run the `WAYS=2` configurations with the model's LRU policy configured to match the RTL's LRU bit.

5 Reusing the Lab V3 Coverage

Test term — Coverage closure. Not a test but the completion criterion of a regression: every bin of the coverage plan must have been hit, merged over all runs, for each parameter set. A passing regression with holes has not shown that the untested features work; closing a hole means adding or steering a test, never deleting the bin.

Test term — Regression. All tests × seeds × `PARAMS` sets of `dv_layered/regress.list`, run by one command with a pass/fail summary. Run before every merge; here it also produces the merged coverage of the closure criterion and must stay green for the Lab V3 environment next to it.

Coverage measures the stimulus and the DUT, not the testbench architecture, so the Lab V3 coverage plan stays valid. Reuse `dcache_cov` from `hw/ip/dcache/dv/cov/dcache_cov.svh` without copying it (add the directory to the include path of the `dv_layered` file list):

- The scoreboard samples `sample_access()` after each comparison at acceptance, with the model's result — which at that point has just been confirmed by the memory-transfer comparison — and `sample_flush()` after each flush comparison.
- The closure criterion of Lab V3 applies unchanged: every bin of the coverage plan hit in each `PARAMS` set, merged over the regression with the same tool (`cache_cg` or the fallback counters and `flow/scripts/cov_merge.py`).
- Compare: how many transactions does each environment need to close coverage with the same tests and seeds? They should be equal if the stimulus is the same; if not, find out why.

If `dcache_cov.svh` depends on a name defined in the Lab V3 `tb_top` (for example the `result_e` type or `IDX_W`), move that definition to a small shared file in `hw/ip/dcache/dv/cov/` included by both environments; the Lab V3 environment must still compile and pass.

6 Bug-Injection Experiment

How good is a testbench? Passing tests do not answer that question: a testbench that checks nothing passes everything.

Test term — Negative / error-injection test. A known bug is put into the RTL on purpose and the same tests are run; the run must *fail*, and the first message must name the right check. It proves that the scoreboard and the assertions actually fire, and here it measures *how early* each environment notices (accepted transactions before the first error).

The direct measurement is to inject known bugs and see whether, where and how fast they are found.

Inject each bug below into the RTL, one at a time, and run the same tests and seeds on both environments. Keep each bug as a patch file in `hw/ip/dcache/dv_layered/bugs/` so the experiment can be reproduced; the patches are never applied on the lab branch.

	T-DV1 environment		Lab V3 environment	
Injected bug	**Detected by**	**Txn #**	**Detected by**	**Txn #**
Write-back uses the new tag instead of the victim's tag				
Dirty bit not cleared after refill				
Refill writes word 1 into word 0				
Write hit does not set the dirty bit				
`misses` counter also counts hits				
Flush skips the last line				

"Detected by" names the comparison, assertion or check that reported first; "Txn #" is the number of accepted CPU transactions at that moment (or "end" for an end-of-test check). For every bug, record also whether a `dcache_sva` assertion fired, and when — the assertions are active in both environments.

Then answer in `doc/verification.md`:

1. Which bugs would the Lab V3 checks have missed or found much later? Explain why for each, from what each check can and cannot see.
2. Is there a bug that neither environment finds with the required tests? Which test or coverage bin would expose it? Add it.
3. What did the model-based scoreboard cost you, in lines of code and in debug time, compared with the three V3 checks? Where was the extra precision worth it?

7 Requirements / Tasks

1. Write `mem_if_vip_pkg` in `dv/vip/mem_if/`: item, subscriber, driver and monitor, parameterized by `AW` and `DW`, with no cache-specific code.
2. Write the `cache_ctrl_if` interface and the `tb_top` of `hw/ip/dcache/dv_layered/tb/` with the Lab V3 DUT setup and the design-assertion bind.
3. Write the stimulus classes (`dcache_txn`, `conflict_txn`, ...) with the Lab V3 knobs and constraints, each overriding `clone()`.
4. Write the generator, the loader, the scoreboard (with the VIP-to-model adapter) and `cache_env`, with the abstract `component` base class.
5. Implement the scoreboard algorithm, including the flush comparison as a set and the performance counter comparison.
6. Implement the end-of-test sequence and the watchdog.
7. Write `base_test` and the six test classes; select them with `+TEST=`.

8. Reuse the Lab V3 coverage; reach coverage closure with `make regress DV_ENV=dv_layered` over the Lab V3 `PARAMS` sets.
9. Run the bug-injection experiment on both environments and fill the table.
10. `make check-all` passes: Labs V3 and T-MOD still run unchanged.

8 Repository and Flow

Files created or changed in this lab:

```
dv/vip/mem_if/
+-- mem_if_vip_pkg.sv         item, subscriber, driver, monitor (includes the .svh)
+-- mem_if_vip.f              file list of the VIP
hw/ip/dcache/
+-- Makefile                  DV_ENV value dv_layered added
+-- dv_layered/
|   +-- dv_layered.f          -f rtl lists, -f VIP, -f dv/model/cache_model, sva, env,
    tb
|   +-- regress.list          tests x seeds x PARAMS sets for this environment
|   +-- tb/tb_top.sv, cache_ctrl_if.sv
|   +-- env/                  dcache_env_pkg.sv (+ .svh): configuration, stimulus
|   |                         classes, control driver/monitor, generator, loader,
|   |                         scoreboard, cache_env
|   +-- tests/                dcache_test_pkg.sv: base_test and one class per test
|   +-- bugs/                 bug01_wb_tag.patch ... bug06_flush_last.patch
+-- dv/cov/                   dcache_cov.svh reused (shared definitions if needed)
+-- doc/verification.md       new section: layered environment, experiment
dv/model/cache_model/             unchanged (Lab T-MOD)
```

The environment is selected with the IP-local variable `DV_ENV` of the `dcache` Makefile, introduced for this purpose. It selects `TB_FLIST` and `TB_TOP`; the default stays `dv` (Lab V3):

```
# hw/ip/dcache/Makefile (excerpt)
DV_ENV ?= dv             # dv (V3) | dv_layered (T-DV1) | dv_uvm (T-DV2)
ifeq ($(DV_ENV),dv_layered)
  TB_FLIST := dv_layered/dv_layered.f
  TB_TOP   := tb_top
endif
```

The two environments must not overwrite each other's build products: check that the build directory of a `dv_layered` run differs from that of a `dv` run with the same `PARAMS`, and fix it in the Makefile if it does not.

The environment file list pulls everything in with nested `-f`; nothing is copied:

```
// hw/ip/dcache/dv_layered/dv_layered.f
-f ${REPO_ROOT}/hw/ip/arb_mem/rtl/arb_mem.f
-f ${REPO_ROOT}/hw/ip/dcache/rtl/dcache.f
${REPO_ROOT}/dv/common/tb_utils_pkg.sv
-f ${REPO_ROOT}/dv/vip/mem_if/mem_if_vip.f
-f ${REPO_ROOT}/dv/model/cache_model/cache_model.f
+incdir+${REPO_ROOT}/hw/ip/dcache/dv/cov
+incdir+${REPO_ROOT}/hw/ip/dcache/dv_layered/env
+incdir+${REPO_ROOT}/hw/ip/dcache/dv_layered/tests
${REPO_ROOT}/hw/ip/dcache/dv/sva/dcache_sva.sv
${REPO_ROOT}/hw/ip/dcache/dv/sva/dcache_bind.sv
${REPO_ROOT}/hw/ip/dcache/dv_layered/tb/cache_ctrl_if.sv
${REPO_ROOT}/hw/ip/dcache/dv_layered/env/dcache_env_pkg.sv
```

```
${REPO_ROOT}/hw/ip/dcache/dv_layered/tests/dcache_test_pkg.sv
${REPO_ROOT}/hw/ip/dcache/dv_layered/tb/tb_top.sv
```

(Use the file list name that Lab T-MOD created for the model package.) Example commands:

```
make -C hw/ip/dcache sim DV_ENV=dv_layered TEST=smoke
make -C hw/ip/dcache sim DV_ENV=dv_layered TEST=conflict SEED=3 WAVES=1
make -C hw/ip/dcache sim DV_ENV=dv_layered TEST=random PARAMS="LINE_WORDS=8 NUM_LINES=8"
     \
     PLUSARGS="+NUM_TXN=20000 +FLUSH_EVERY=500"
make -C hw/ip/dcache regress DV_ENV=dv_layered
make -C hw/ip/dcache regress                        # Lab V3 environment, unchanged
# bug injection, one bug at a time
git apply hw/ip/dcache/dv_layered/bugs/bug03_refill_word.patch
make -C hw/ip/dcache sim DV_ENV=dv_layered TEST=random SEED=1
make -C hw/ip/dcache sim DV_ENV=dv         TEST=random SEED=1
git apply -R hw/ip/dcache/dv_layered/bugs/bug03_refill_word.patch
```

On the Synopsys cloud: The commands above use the cloud defaults (`SIM=vcs`). Waves: `WAVES=1` then `make waves` (Verdi with FSDB). Code and functional coverage: `make regress DV_ENV=dv_layered COV=1`, then `make cov DV_ENV=dv_layered PARAMS="..."` (VCS `-cm` and URG; the covergroups `cache_cg`/`flush_cg` appear under "Groups").

9 Deliverables

The merge request (branch `lab/tdv1-layered-tb`, tag `tdv1-done`) contains:

- `dv/vip/mem_if/` and `hw/ip/dcache/dv_layered/` complete, the updated `dcache` Makefile, the six bug patches; `make check-all` passes.
- A new section in `hw/ip/dcache/doc/verification.md` containing:
  - a class diagram: inheritance (component, VIP classes, stimulus transactions, tests) and ownership (which object creates and holds which);
  - the communication choices and their reasons;
  - the scoreboard algorithm, the adapter between VIP and model, and the mapping from Lab V3 checks to the new comparisons;
  - the end-of-test sequence;
  - the regression summary and the merged coverage report;
  - the bug-injection table for both environments and the answers.
- A `dv/vip/mem_if/README.md`: what the VIP does, its parameters, how to instantiate a driver and a monitor, and how to subscribe to a monitor.
- `hw/ip/dcache/README.md` updated with the `DV_ENV` values and how to run each environment.

10 Assessment

Criterion	Weight
Testbench architecture: layers, components, communication, abstract component	20%
Reusable `mem_if` VIP: independence from the cache, documentation	15%
Scoreboard and connection of the Lab T-MOD model to the RTL	20%
Clean end of test, watchdog, reporting	10%
Tests, blueprint pattern and object-oriented quality	10%
Coverage reuse and closure regression	10%
Bug-injection experiment and comparison with Lab V3	15%
Total	100%

11 Review Questions

1. Why does the scoreboard compare memory transactions at the moment the CPU request is accepted, and not when they occur?
2. Why are flush write-backs compared as a set and not as a list?
3. Why is the generator-to-driver mailbox bounded? What goes wrong with an unbounded one when the test ends?
4. Why do monitors call the scoreboard through a subscriber interface instead of holding a scoreboard handle?
5. A derived transaction class forgets to override `clone()`. What does the generator produce, and how would you notice?
6. Your model and your RTL agree on every transaction. How do you know the model is right? (Hint: what did Lab T-MOD do?)
7. Why must the `mem_if` VIP not import `cache_model_pkg`, and the model not import the VIP? Where does the conversion live?
8. The model does not predict timing. Which parts of this environment check timing instead?
9. Which injected bug was hardest to detect, and why?
10. Which part of this testbench do you expect UVM (Lab T-DV2) to replace, and which part to keep unchanged?

12 References

- IEEE Std 1800-2017: Clause 8 (classes), Clause 15 (interprocess synchronization: semaphores, mailboxes, events), Clause 25.9 (virtual interfaces).
- Chris Spear and Greg Tumbush, *SystemVerilog for Verification*: chapters on testbench basics, advanced OOP and testbench guidelines (the blueprint pattern), and threads and interprocess communication.
- Janick Bergeron, *Writing Testbenches Using SystemVerilog*: layered testbench architecture.

- Verification Academy: SystemVerilog OOP and testbench architecture material.
- OpenTitan: DV methodology and the `hw/dv/sv` shared verification IP directory, as an example of protocol VIP separated from block environments.